DAILY DATA DEEP DIVES · THE LATEST NEWS, FULLY EXPLAINED

Storage Wars

MongoDB searches text it can't read, and Spark streams get to change their minds

Issue #10 | Wed, 14 Oct 2026 | ⏱️ ~40 min read | Level 3/5

🗄️ **Today's two stories**

1. **MongoDB's Queryable Encryption can now run prefix, suffix and substring searches on encrypted strings in production.** Your app can find every patient whose diagnosis contains "diabet" while the database server only ever sees ciphertext. It shipped as generally available with MongoDB 9.0, a year after a public preview in 8.2, and MongoDB announced the GA on 6 October. The catch is the bill: one substring-searchable field can cost up to 236 index entries per document. We explain how searchable encryption works, why substring search is so expensive, what the server can still learn, and how to size it before you turn it on.
2. **Databricks made "on-demand state repartitioning" generally available on 13 October.** For as long as Spark Structured Streaming has existed, the number of partitions a stateful streaming job started with was frozen into its checkpoint forever. Changing it meant throwing away all the job's state. Now you change one setting and restart, and Databricks moves the state for you. We explain why the partition count was frozen, how Flink avoided the problem from the start, and what to check before you resize a production stream.
3. **On the radar:** 7 more things that happened, listed at the end.

STORY 1 MongoDB learns to search text it can't read

MONGODB 9.0 · QUERYABLE ENCRYPTION · ANNOUNCED 29 SEP 2026 · GA BLOG 6 OCT · ~19 MIN READ

Picture a hospital database where a nurse types "diabet" into a search box and gets back every patient with "diabetes" or "diabetic" in their chart. Nothing special, except that the database server running that search can't read a single diagnosis. It never sees the word "diabet" either. The data sits on disk, in memory and in backups as random-looking bytes, and the keys to unlock it live only in the hospital's application. As of MongoDB 9.0, that's a supported production feature. It's also one of the most expensive things you can ask a database to index, and the reason why is a nice lesson in how encrypted search really works.

WHAT HAPPENED

On **29 September 2026**, MongoDB launched MongoDB 9.0 at its Investor Day in New York. The press release said 9.0 "expands industry-first Queryable Encryption with support for prefix, suffix, and substring searches on encrypted data, allowing organizations to query sensitive information without ever decrypting it."[1] A week later, on **6 October**, a MongoDB blog post made the status explicit: prefix, suffix and substring queries are now generally available for production workloads. Until then, only equality and range queries were GA.[2]

Here's what changed, from the 9.0 documentation:

- Three new query types for encrypted `string` fields: `prefix`, `suffix` and `substring`.[3]
- Four new aggregation expressions that use them: `$encStrStartsWith`, `$encStrEndsWith`, `$encStrContains` and `$encStrNormalizedEq`. "All other aggregation expressions return an error if issued against encrypted fields."[3]
- Substring search is for short fields. The field can be at most **50 characters** (`strMaxLength` is "Integer from 2-50, inclusive"), and the search term must be **2 to 6 characters** long.[4]
- Case and accent handling are chosen per field, with the required `caseSensitive` and `diacriticSensitive` options.[4]
- It's available in MongoDB Atlas, Enterprise Advanced and Community Edition at no extra cost, according to MongoDB's blog.[2] There's a nuance in the docs, covered below: on Community, you have to do the encryption calls yourself.[5]

The preview that came before it isn't compatible. MongoDB 8.2 had shipped these as `prefixPreview`, `suffixPreview` and `substringPreview` using an algorithm called `text`. In 9.0 those "are deprecated and removed. Use `prefix`, `suffix`, or `substring` query types with the `string` algorithm instead."[6] The drivers moved first: the Node.js driver 7.5.0 (7 July 2026) marked the feature GA for MongoDB 9.0, renamed its `TextOpts` API to `StringOpts` and the `TextPreview` algorithm to `String`, and deprecated the preview names.[7]

THE BACKGROUND YOU NEED

Three places your data can be exposed

Databases have long encrypted data in two of the three places it lives. **Encryption in transit** *≈ encrypting data as it travels over the network, usually with TLS, so someone snooping on the wire sees only ciphertext* protects it on the wire. **Encryption at rest** *≈ encrypting the database files and backups on disk, so a stolen disk or backup is unreadable without the key* protects it on disk. But neither helps once the data is inside the running server. To answer `WHERE ssn = '...'`, the server decrypts pages into memory and compares plaintext. Anyone with enough access to the server, such as a database admin, a cloud operator, an attacker with a memory dump or someone reading logs, can see your users' data.

Encryption in use *≈ keeping data encrypted even while the database is processing queries over it, so the server never handles plaintext* closes that last gap. MongoDB's version is client-side: your application's driver encrypts sensitive fields before they leave the process, and only the driver can decrypt them. The docs list what that defends

against: "Direct access to encrypted fields by a database superuser", "reading the server's memory", "Capture of encrypted fields over an insecure network", "reading database or backup files", and "Frequency analysis attacks".[8]

Keys come in two layers. Each encrypted field uses a **data encryption key (DEK)** ≈ *the key that actually encrypts field values; MongoDB stores DEKs in a key vault collection, themselves encrypted*, and the DEKs are wrapped by a **customer master key (CMK)** ≈ *the top-level key, held in a key management service such as AWS KMS, Azure Key Vault or GCP KMS, that encrypts the data encryption keys* held in a **KMS** ≈ *key management service: a system, usually a cloud service or hardware module, that stores master keys and performs encryption with them without handing them out*. MongoDB says that in production "you must use a remote Key Management System (KMS) to store your encryption key."[8]

The old trick: deterministic encryption

Searching encrypted data is easy if you're willing to give something up. MongoDB's older feature, **CSFLE** ≈ *Client-Side Field Level Encryption: MongoDB's earlier encryption-in-use feature, which supports equality queries only on deterministically encrypted fields*, can encrypt a field **deterministically** ≈ *so that the same plaintext always produces the same ciphertext under the same key*. The docs put it plainly: "With deterministic encryption, a given input value always encrypts to the same output value."[9] To find `ssn = '123-45-6789'`, the driver encrypts the search value and the server looks for identical ciphertext, with a normal index.

The cost is that identical values look identical on disk. If one encrypted "blood type" value appears in 38% of documents, an attacker with a copy of the database and a public table of blood-type frequencies can make a very good guess at what it is. That's a **frequency analysis attack** ≈ *inferring encrypted values by matching how often each ciphertext appears against known real-world frequencies, like breaking a substitution cipher by spotting the most common letter*, the same idea that breaks newspaper cryptograms.

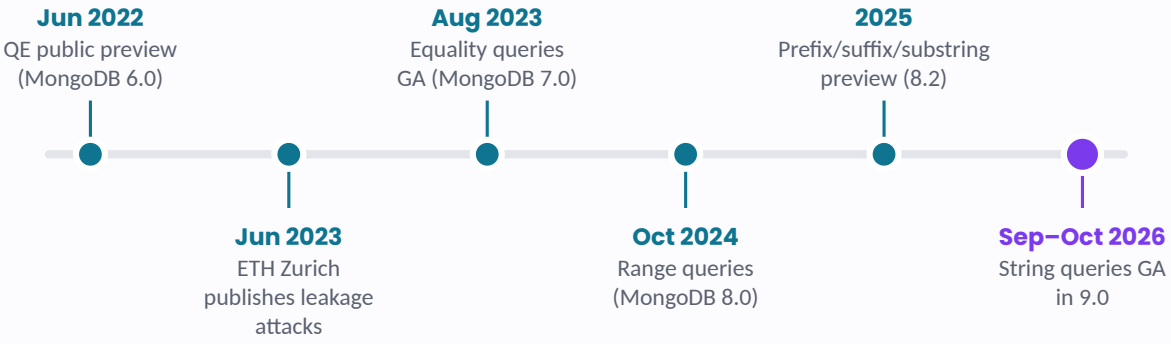
Queryable Encryption: random ciphertext, searchable anyway

Queryable Encryption (QE) ≈ *MongoDB's encryption-in-use feature (preview 2022, GA 2023) that uses randomized encryption plus encrypted search tokens, so equal values don't look equal on disk but can still be queried* takes the harder road. "The encryption algorithm for Queryable Encryption uses randomized encryption based on structured encryption, which produces different encrypted output values from the same input."[9] Two patients with diabetes get two completely different ciphertexts.

To still be able to search, QE borrows from an academic field called **structured encryption** ≈ *a branch of cryptography for encrypting data structures (like indexes or dictionaries) so that a server can answer specific queries on them using tokens, while learning as little as possible*. The driver stores, next to the randomly encrypted value, a set of cryptographic **tags** ≈ *short pseudorandom values derived from a field value and a secret key; the server can match tags against query tokens without learning the underlying value*. At query time it sends a **token** ≈ *a value derived from the search term and secret key that lets the server find matching tags for exactly this query, and nothing else* derived from the search term. The server can match tokens to tags, but neither tells it what the value is.

MongoDB runs a cryptography research group whose published work, much of it by Seny Kamara and Tarik Moataz, covers structured encryption and its leakage, including a technical report that set out to design "the first encrypted document database scheme" that is "practical with respect to real-world constraints".[10]

Queryable Encryption, from preview to substring search



Sources: ETH Zurich paper, 7.0 GA release, 7.0 compatibility notes, 8.0 GA release, 8.2 release notes, GA blog.

The road there has had potholes. When equality queries went GA in 7.0, the "improvements in the GA make it incompatible with the Queryable Encryption Public Preview", and collections built on the 6.x preview had to be recreated or decrypted and reloaded.[11] The 2023 GA press release already promised that range, prefix, suffix and substring queries would follow.[12] Range arrived with MongoDB 8.0 in October 2024.[13] String search took two more years, and the same compatibility break has happened again: the 8.2 preview docs warned that GA "will be incompatible with the preview feature" and you "will have to drop any collections that enable these queries."[14]

HOW IT WORKS

The moving parts

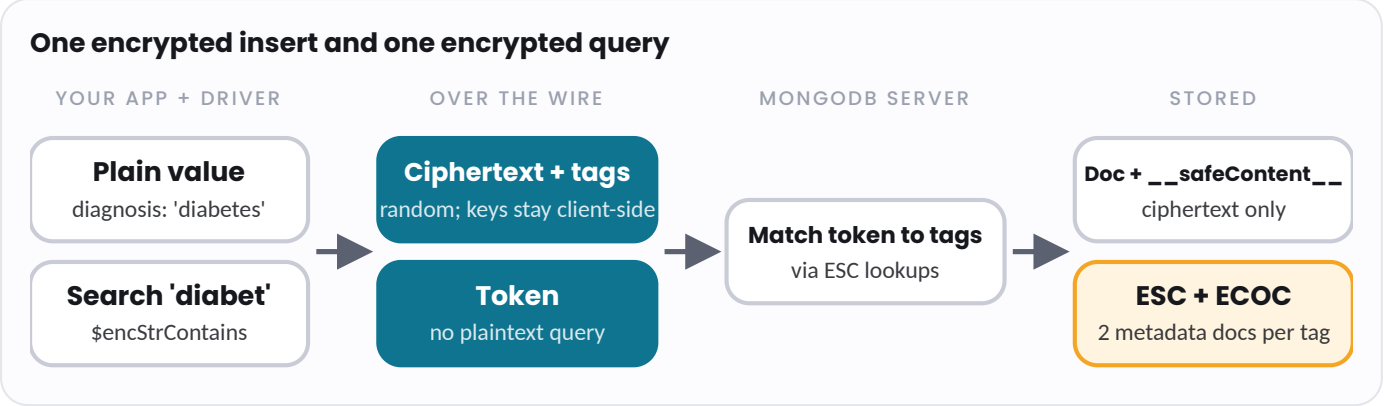
When you create a QE collection, MongoDB adds two internal **metadata collections** *≈ hidden collections that QE creates next to yours, named enxcol_.NAME.esc and enxcol_.NAME.ecoc, holding the encrypted bookkeeping that makes search possible* called `enxcol_.<collectionName>.esc` and `enxcol_.<collectionName>.ecoc`, and adds a `__safeContent__` field to each document that holds the tags.[8] The docs don't spell out the names, but the ETH Zurich analysis described below calls the **ESC** *≈ QE's metadata collection of encrypted counters, which the server uses to find the documents matching a token* an Encrypted State Counter store, and the **ECOC** *≈ QE's metadata collection of tokens from recent writes, kept so the server can later compact the ESC* the place where write tokens wait for **compaction** *≈ a periodic clean-up in which the server merges the ECOC's write records into the ESC and then clears the ECOC*.[17] The docs are blunt about them: "Do not modify these data structures or query results will be incorrect and security could be impacted."[8]

A write goes like this:

1. Your driver sees a document with a field marked encrypted in the collection's `encryptedFields` schema.
2. It encrypts the value with randomized encryption, so equal values produce different ciphertext.
3. For each kind of query the field supports, it derives the right **tags** using secret keys only the client has.
4. It sends the ciphertext and tags to the server. The server stores the document and writes "two metadata documents per tag, one to the ESC and one to the ECOC".[16]

A query runs the same idea in reverse. The driver turns your search term into a token and sends it in place of the plaintext; the server uses the ESC to find the documents whose tags match; the encrypted documents come back; and the driver decrypts them.

An independent 2023 analysis by ETH Zurich cryptographers described the original 2022 design in detail. Each tag works roughly like a counter: the server keeps an encrypted count per value, and a search enumerates the counters from 1 up to the maximum to find every document holding that value.[17] That's why writes to the same value can collide, and why QE has a knob called **contention** *≈ a QE setting that spreads inserts of the same value across several counters (default 8) so concurrent writes don't fight over one; reads must then check all of them*. Its default is 8. "Higher contention improves the performance of insert and update operations on low cardinality fields, but decreases find performance."[4] (The design has changed since 2022, so treat the ETH description as background on the idea, not a spec of 9.0.)



Why a substring index is so big

Equality is cheap: one value, one tag. "Fields indexed for equality queries generate one tag, `T=1`."[16]

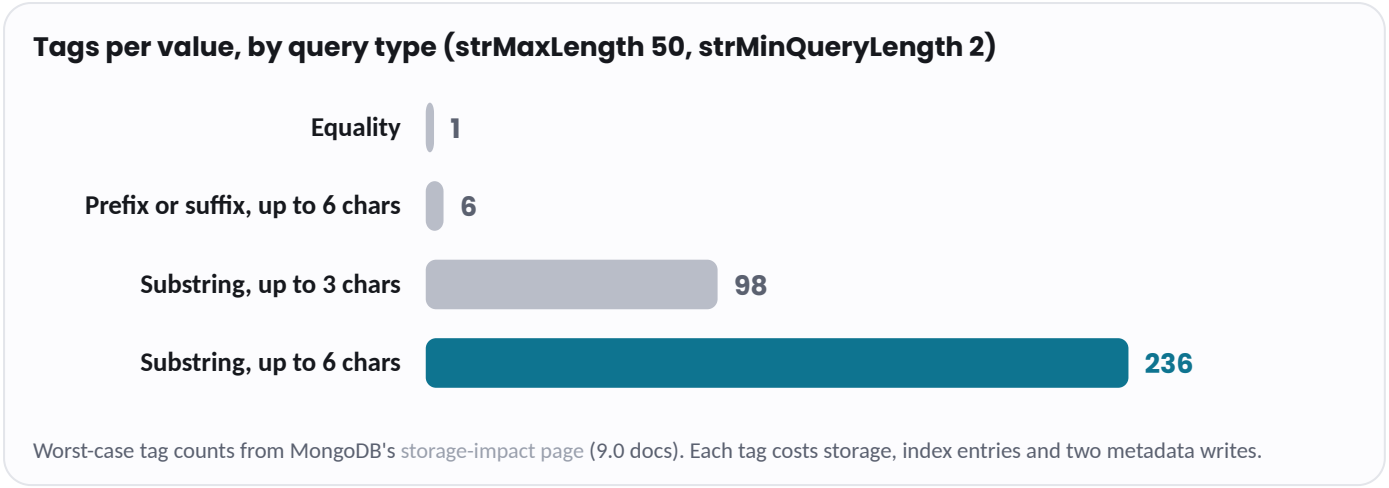
Substring search can't work that way. There's no way to turn the search term "diabet" into a token that matches the tag for "diabetes" unless, at insert time, the driver already made a tag for "diabet". So it does: it makes a tag for every possible substring the field might later be searched for. With the GA limits, that's every 2-character slice, every 3-character slice, and so on up to the configured maximum query length, starting at every position in the string.

MongoDB publishes the worst-case formulas:[16]

- Prefix or suffix: `T = 1 + (strMaxQueryLength - strMinQueryLength + 1)`
- Substring: `T = 1 + (strMaxQueryLength - strMinQueryLength + 1) * (2 * strMaxLength + 2 - strMaxQueryLength - strMinQueryLength) / 2`

For the widest allowed substring setup, a 50-character field searchable with 2 to 6 character terms, that's `1 + 5 × 94 / 2 = 236` tags per value.[16] Prefix search is linear in the query-length range; substring search is that range multiplied by the field length.

The formulas are for the *maximum*, as in "the maximum number of tags a value can generate".[16] Notice they're written in terms of `strMaxLength`, not the length of the actual string. The docs don't say why, but it's consistent with not wanting the number of tags to reveal how long each value is.



Tags aren't free. MongoDB's sizing formulas charge about 255 bytes of document storage and 67 bytes of index storage per tag, plus 52 bytes of memory, each with a 1.2x safety factor on disk.[16] Plugging a 20-byte value into those worst-case formulas gives a rough per-document cost for a collection with one encrypted field:

Field configured for	Tags	Disk per doc (worst case)	10M docs	Memory, 10M docs
Equality	1	~1.3 KB	~13 GB	~0.6 GiB
Prefix, 1–10 chars	11	~5.2 KB	~52 GB	~5.5 GiB
Substring, 50 chars, 2–6	236	~92 KB	~920 GB	~115 GiB

Our arithmetic, using the formulas on MongoDB's storage-impact page; real usage "will likely have a smaller disk storage and memory impact than calculated. These formulas guard against the worst case."[16]

Writes scale the same way, because every tag means an ESC write and an ECOC write: "write throughput scales with the tag count in the same way as storage."[16] An independent test of the 8.2 *preview* (one developer, labelled AI-assisted, on 8.2.12 Community with PyMongo) found substring inserts took 22.9 ms per document versus 1.16 ms for an equality-encrypted field, and storage per document rose from 639 bytes to about 50.6 KB, including metadata. A `$encStrContains` query on 500 documents took 25.3 ms, against 1.0 ms for a plaintext `$regex`.[18] That was the preview's `text` algorithm, not the GA `string` one, so take the exact numbers as a rough sense of scale.

MongoDB's own advice is cautious: "avoid enabling substring queries on more than one field. Enable them only if prefix and suffix queries can't meet your needs, and the collection contains fewer than 50 million documents."[16]

Configuring a field

The schema lives in `encryptedFields` when you create the collection. Each field gets a `queries` entry. You can configure a field for "only one of the equality or range query types", "can enable both `prefix` and `suffix` on the same field, but you can't combine `substring` with another query type."[15]

Two encrypted string fields (illustrative)

Prefix + suffix on an ID

```
{
  path: "nationalId",
  bsonType: "string",
  queries: [
    { queryType: "prefix",
      strMinQueryLength: 2,
      strMaxQueryLength: 4,
      caseSensitive: false,
      diacriticSensitive: false },
    { queryType: "suffix",
      strMinQueryLength: 4,
      strMaxQueryLength: 4,
      ... same flags ... }
  ] }
```

Substring on a short label

```
{
  path: "diagnosisCode",
  bsonType: "string",
  queries: {
    queryType: "substring",
    strMaxLength: 30,
    strMinQueryLength: 3,
    strMaxQueryLength: 5,
    caseSensitive: false,
    diacriticSensitive: false
  } }
```

Our sketch within the 9.0 limits (substring: strMaxLength 2–50, query length 2–6; prefix/suffix: min ≥ 1). Option names from the 9.0 docs; check your driver's docs for the exact shape of multi-type queries. caseSensitive and diacriticSensitive must match across types on one field.

The two flags at the bottom are required for these types, and they decide what counts as a match: `caseSensitive: false` for case-insensitive matching, `diacriticSensitive: false` to treat "é" and "e" alike. If a field has more than one query type, "this value must be identical for all of them."[4]

Queries use `$expr` inside an aggregation `$match`. This is the documented example for prefix search:[19]

```
db.collection('MyCollection').aggregate([
  { $match: { $expr: {
      $encStrStartsWith: {
        input: '$employeeFirstName',
        prefix: 'Sam'
  } } } }
])
```

The search string has to fall inside the configured length range ("Attempting to query a shorter string returns an error"), and matching treats whitespace literally: "Searches match whitespace characters", "Tokenization delimiters aren't supported."[4,19] This is not full-text search; there's no stemming, ranking or word splitting.

Automatic vs explicit encryption

There are two ways to drive all this. With **automatic encryption** *≈ the driver reads the collection's encryption schema and encrypts fields and query values for you; in MongoDB this needs Enterprise or Atlas*, you write normal queries and the driver rewrites them. With **explicit encryption** *≈ your code calls the driver's ClientEncryption API to encrypt each value and*

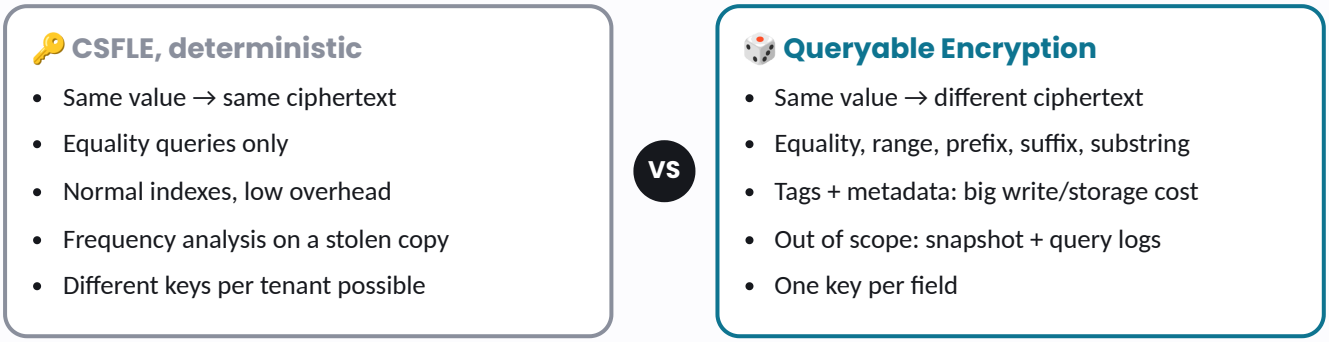
query term itself; available on Community Server too, you do it by hand. "Automatic decryption is available in MongoDB Community Server. Automatic encryption requires MongoDB Enterprise or MongoDB Atlas." On Community you set `bypassQueryAnalysis` to true and use the `string` algorithm with the required parameters yourself.[5] So "available in Community" is true, with more code.

WHAT THE SERVER CAN STILL LEARN

No practical encrypted-search scheme hides everything, and QE doesn't claim to. The amount it reveals is called **leakage** *≈ what an encrypted-search system unavoidably reveals to the server, such as which stored records matched a query or which queries repeat, even though values stay encrypted*, and the docs' threat table lists what's left: network snooping "Reveals operation metadata"; disk recovery "Reveals *size of database* and operation metadata".[8]

The important line is about scope. QE "defends against data exfiltration, not against adversaries with persistent access to an environment, or those who can retrieve both database snapshots and accompanying query transcripts/logs." It goes on: "This is **especially** true for range queries, even if only a small number of query transcripts or logs are retrieved."[8]

That boundary has a history. In June 2023, ETH Zurich researchers Zichen Gui, Kenneth Paterson and Tianxin Tang published a security analysis of the 2022 preview. They found that MongoDB's own logs, needed for normal operation, recorded which metadata entries each query touched, which reveals which documents share a value. Using a snapshot of documents plus those logs and a similar public dataset, their attack recovered 40% to 80% of field values in their simulations, and they judged the contention factor "offers limited protection".[17] They reported it to MongoDB in October 2022 and concluded that "a major rethink of the QE architecture may be needed".[17] MongoDB's docs now say that, compared with CSFLE, "private querying goes a step further and redacts logs and metadata to scrub information around the query's existence."[9] The docs don't say exactly which of the paper's findings that addresses.



From MongoDB's QE vs CSFLE guide and QE features page.

Substring search adds its own question, which the GA blog doesn't answer: the post describes the scheme only as open-source, independently verifiable structured encryption, without a construction or a leakage profile for the new types.[2] Common sense says a search for a 2-character term will match many more documents than a 6-character one, and the server sees how many documents come back. The 2–6 character limit and 50-character cap keep the scheme manageable, but if you need to explain QE's guarantees to an auditor, you'll want MongoDB's technical paper rather than the marketing page.

WHAT IT MEANS FOR YOU

If you store regulated identifiers or short labels, this is the most useful QE release since equality search. The blog's examples are realistic: finding users by last name or email prefix, suffix search on the last four digits of an SSN, National Insurance Number or Aadhaar number, and case IDs starting with "NYC-".[2] Most of those are prefix or suffix jobs, and those are cheap. A suffix-only field searchable on exactly four characters needs `1 + 1 = 2` tags.

Treat substring as a last resort. Before enabling it, run the storage-impact formulas on your real field lengths and document count, and pick the narrowest settings that work: the smallest `strMaxLength`, the tightest query-length range. Then load-test inserts, because each value costs up to hundreds of metadata writes. MongoDB also points to an agent skill, `mongodb-qe-size-estimation`, that does the arithmetic for you.[16]

Plan your schema up front. "Changing the increases requires you to re-create the collection,"[16] and adding encrypted fields "requires rebuilding metadata collections and indexes".[9] If you tried the 8.2 preview, those collections must be dropped and rebuilt with the GA types on 9.0.[6,14]

Upgrade everything together. You need MongoDB 9.0, a driver release that speaks the GA `string` algorithm (for Node.js, 7.5.0 or later), and, if you use automatic encryption, a matching shared library.[7,6]

💥 Watch out for

- **The docs contradict themselves on limits.** The 9.0 reference says substring queries need `strMaxQueryLength` of 2–6, but the example on the schema page uses `strMinQueryLength: 1` and `strMaxQueryLength: 20`, left over from the preview.[4,15] Trust the reference table.
- **No `$regex`, no `null`.** "Queries that compare an encrypted field to `null` or a regular expression always throw an error."[3] Existing `$regex` code needs rewriting to `$encStr*` expressions.
- **Encrypted fields count against the 16 MB document limit,** including their metadata.[16]
- **Keep your logs private anyway.** QE's guarantees stop at "database snapshots and accompanying query transcripts/logs".[8]
- **One QE scheme per collection.** You can't mix QE and CSFLE in the same collection.[9]

WHAT HAPPENS NEXT

The open questions are practical: how much the GA `string` algorithm costs compared with the preview, which nobody independent has measured yet, and whether MongoDB publishes a leakage analysis for the new query types the way it has for equality. If you're evaluating it, measure on your own data first.

📖 Words from this story

Encryption in transit: encrypting data on the network, usually with TLS.

Encryption at rest: encrypting database files and backups on disk.

Encryption in use: keeping data encrypted even while the database processes queries on it.

Data encryption key (DEK): the key that encrypts field values, itself stored encrypted in a key vault collection.

Customer master key (CMK): the top-level key in a KMS that encrypts the DEKs.

KMS: key management service that stores master keys and does encryption without handing them out.

CSFLE: Client-Side Field Level Encryption, MongoDB's earlier feature; equality queries on deterministically encrypted fields.

Deterministic encryption: the same plaintext always produces the same ciphertext.

Frequency analysis attack: guessing encrypted values from how often each ciphertext appears.

Queryable Encryption (QE): MongoDB's randomized, searchable encryption-in-use feature.

Structured encryption: cryptography for encrypted data structures that answer specific queries via tokens.

Tags: pseudorandom values stored with an encrypted field, derived from the value and a secret key.

Token: a value derived from a search term that lets the server find matching tags for that query only.

Metadata collections: QE's hidden `enxcol_.NAME.esc` and `enxcol_.NAME.ecoc` collections.

ESC: QE's metadata collection of encrypted counters, used to find documents matching a token.

ECOC: QE's metadata collection of tokens from recent writes, kept for compaction.

Compaction: merging the ECOC's write records into the ESC, then clearing the ECOC.

Contention: a QE setting (default 8) that spreads writes of the same value across counters; reads check them all.

Automatic encryption: the driver encrypts fields and queries from the schema for you (Enterprise/Atlas).

Explicit encryption: your code calls the encryption API for each value and query term (also on Community).

Leakage: what an encrypted-search system reveals to the server despite encryption.

STORY 2 Spark streaming jobs can finally change their partition count

DATABRICKS · SPARK STRUCTURED STREAMING · GA 13 OCT 2026 · PREVIEW SINCE JUN 2026 · ~16 MIN READ

Somewhere in your company there's probably a streaming job running with 200 partitions. Nobody chose 200. It's Spark's default, and it was baked in on the day someone first started the job, maybe three years ago when traffic was a tenth of today's. Since then the cluster has grown, the costs have grown, and the number has stayed exactly where it was, because changing it meant deleting the job's memory and starting over. Yesterday, Databricks made the fix generally available. It's one config setting, and the story of why it took this long says a lot about how stateful streaming works.

WHAT HAPPENED

On **13 October 2026**, the Databricks platform release notes announced: "On-demand state repartitioning is now generally available." You "can resize the number of shuffle partitions for a stateful Structured Streaming query or Lakeflow pipeline" while keeping its checkpointed state.[20]

It went into Public Preview on 10 June 2026 with Databricks Runtime 18, which "now supports changing the number of shuffle partitions for stateful queries without losing checkpoint state."[21] Databricks' engineering blog explained it in detail on 14 September.[22]

The facts, from the docs:

- **Requirements:** Databricks Runtime 18 or above, and "The query must use the RocksDB state store provider", which "On DBR 17.3 or above" is the default.[23]
- **How:** stop the query, set `spark.sql.streaming.stateStore.partitions` to the number you want, and restart it from the same checkpoint.[23]
- **Where:** plain Structured Streaming, and **Lakeflow pipelines** *≈ Databricks' declarative pipeline framework (built on Spark Declarative Pipelines), where you define tables and flows and the platform runs the streaming jobs* through `spark_conf` on a flow or table.[23]
- **What:** the blog lists aggregations, stream-stream joins, deduplication, sessionization and **transformWithState** *≈ Spark's newer API for custom stateful streaming logic, where your code reads and updates per-key state directly*.[22]

It's a Databricks feature. The blog doesn't mention an upstream Apache Spark change, and open-source Spark's docs still say the partition count can't change after a stateful query has run.[22,24]

THE BACKGROUND YOU NEED

Stateless and stateful streams

Spark Structured Streaming *≈ Apache Spark's stream-processing engine, which runs a streaming query as a series of small batch jobs over newly arrived data* runs your query as a sequence of **micro-batches** *≈ small batch jobs, each processing*

the data that arrived since the previous one; Structured Streaming's default execution model: every few seconds it takes the new records from the source (say, a Kafka topic), processes them and writes the result.

Many streaming jobs are **stateless** *≈ each record is processed on its own, with no memory of earlier records, like filtering or reformatting*: filter, reshape, write. A stateless query "only tracks information about which rows have been processed from the source to the sink."[25] Others must remember. Counting clicks per user per 5-minute window, removing duplicate events, joining two streams, detecting user sessions: all need to keep data between batches. That remembered data is called **state** *≈ the intermediate data a streaming query keeps between micro-batches, such as running counts per key or buffered rows waiting for a join*, and "Stateful operations include streaming aggregation, `distinct`, `dropDuplicates`, stream-stream joins" and custom stateful code.[25]

State can't live only in memory, because jobs crash and get restarted. So Spark saves it to a **checkpoint** *≈ a directory in durable storage (like S3) where a streaming query records its progress through the source and a copy of its state, so a restarted query resumes exactly where it stopped*, along with how far through the source the query has read. A restarted query reads its checkpoint and carries on as if nothing happened.

Why state is split into partitions

Big streams don't fit on one machine, so Spark splits the work. Before a stateful step, it **shuffles** *≈ redistributes rows across the cluster over the network so that all rows with the same key end up in the same task* the data so that every row for a given key (say, user 42) lands in the same task. The number of buckets is the **shuffle partition count** *≈ how many buckets Spark splits data into during a shuffle, set by spark.sql.shuffle.partitions; the default is 200*, `spark.sql.shuffle.partitions`, and a key's bucket is chosen by hashing it: roughly `hash(key) mod numPartitions`.

Each bucket owns the state for its keys. On Databricks, each partition's state lives in its own **RocksDB** *≈ an embedded key-value storage engine built on an LSM tree, that keeps data on local disk* instance, stored within the checkpoint.[22] (RocksDB flushes changes to local disk and syncs files to the checkpoint location; with **changelog checkpointing** *≈ writing only the state records that changed in each micro-batch to the checkpoint, instead of re-uploading snapshot files every time*, on by default in DBR 17.3+, "only writes records that have changed since the last checkpoint."[26])

The freeze

Now you can see the problem. If user 42 hashes into partition 17 out of 200, their running count is in partition 17's RocksDB. Restart with 300 partitions and user 42 now hashes into, say, partition 142. Partition 142's state store has never heard of them, and the count starts at zero. Almost every key moves, and almost all of the state is now in the wrong place.

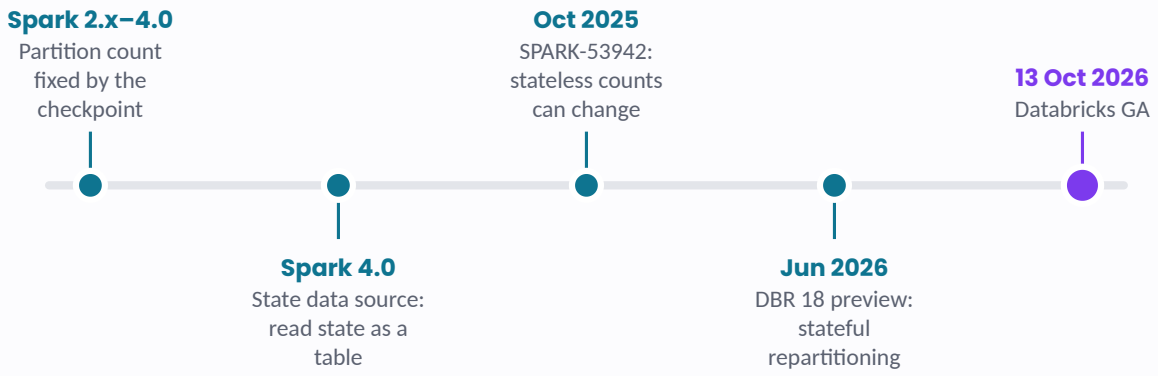
So Spark refused. Its programming guide, as of Spark 4.2, lists `spark.sql.shuffle.partitions` among the configurations that "are not modifiable after the query has run. To change them, discard the checkpoint and start a new query." The reason given: "state is partitioned via applying hash function to key, hence the number of partitions for state should be unchanged."[24] In practice, Spark doesn't even complain; it ignores you.

Databricks' docs: "Changing `spark.sql.shuffle.partitions` has no effect on a streaming query that already has a checkpoint."[25] The blog's phrasing: the query "quietly ignores your new value."[22]

That left two bad options for a job with the wrong partition count:

- **Overprovision forever.** Keep 200 partitions on a tiny stream (200 tasks per micro-batch, each with its own state files to sync) or keep too few on a huge one (big, slow partitions).
- **Throw away the state.** Start with a fresh checkpoint and either lose the running aggregates or rebuild them by replaying source data, if the source still has it.

Spark streaming's frozen partition count, thawing



Sources: Spark guide, state data source, SPARK-53942 (fix version 4.1.0), DBR 18 notes, Databricks release notes.

The stateless half was fixed first, in the open. In SPARK-53942, Jungtaek Lim, who reported and fixed it, noted that "once the streaming query runs, there is no way but discard the checkpoint to change the number of shuffle partitions", even for shuffles that have no state at all, such as **stream-static joins** *≈ joins between a stream and an ordinary, non-streaming table, which need no state between batches*. The fix shipped in Spark 4.1.0 (resolved 25 October 2025).[27] Databricks Runtime 18 is "powered by Apache Spark 4.1.0", and its notes say you can change shuffle partitions in stateless streaming queries.[21] The stateful half is the hard one, and that's what went GA yesterday.

How Flink sidestepped this

Not every engine had the problem. Apache Flink splits keyed state into a fixed number of **key groups** *≈ Flink's fixed-size slices of keyed state; keys hash into key groups, and whole key groups are assigned to parallel tasks, so rescaling just reassigns groups*, and "there are exactly as many Key Groups as the defined maximum parallelism."[28] Keys hash into key groups, which never change, and key groups are dealt out to however many parallel tasks you run today. Rescaling means handing whole key groups to different tasks, with no rehashing of individual keys.

The trade-off is a ceiling chosen on day one. Flink's default maximum parallelism is "roughly operatorParallelism + (operatorParallelism / 2)", with "a lower bound of 128 and an upper bound of 32768", and "Changing the maximum parallelism explicitly when recovery from original job will lead to state incompatibility."[29] Same freeze, just set much higher.

🔥 Spark: hash into partitions

- Key → hash mod numPartitions
- One state store per partition
- Change count = every key moves
- Databricks now rehashes state on restart

```
200 → 300: rewrite all state
```

VS

🐿️ Flink: hash into key groups

- Key → key group (fixed count)
- Groups dealt out to tasks
- Rescale = reassign whole groups
- Max parallelism itself is frozen

```
4 → 8 tasks: move groups, not keys
```

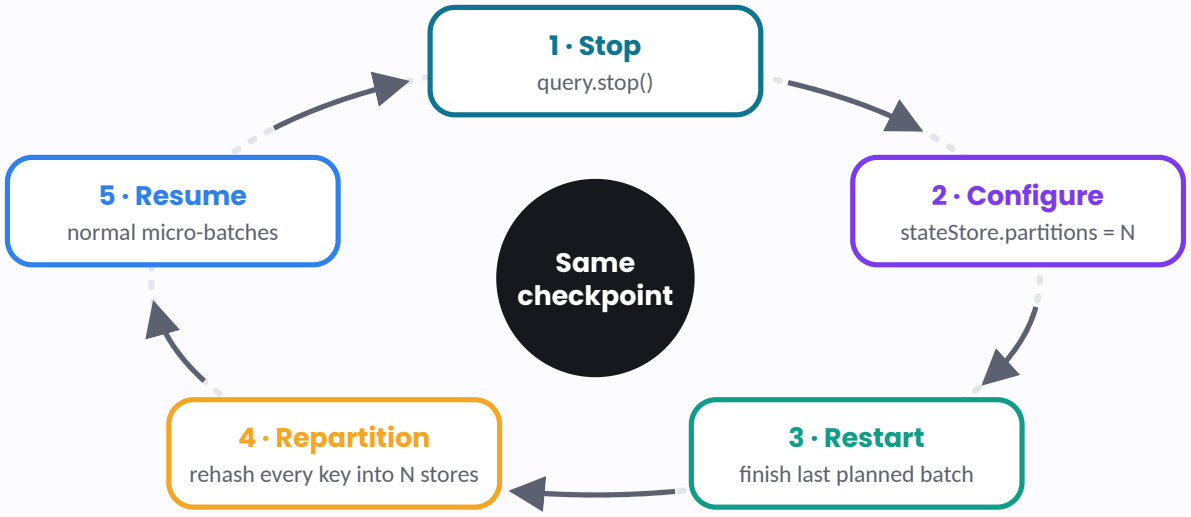
From the Spark guide, Databricks blog and Flink docs on stateful processing and parallelism.

HOW IT WORKS

Databricks took the direct route: when you ask for a new partition count, move every key.

Here's the sequence from the docs and blog. You stop the query and set the new count in `spark.sql.streaming.stateStore.partitions`. For stateful queries, this setting takes precedence over `spark.sql.shuffle.partitions`. On restart, the query finishes the last planned micro-batch, runs a repartition operation that moves the state into the new number of partitions, and then resumes processing.[23] The blog describes it as a one-time step that rehashes every key into its new partition.[22]

A resize, step by step



In code, it's three lines. This is the documented pattern:[23]

```
query.stop()
spark.conf.set("spark.sql.streaming.stateStore.partitions", "<numPartitions>")
query = df.writeStream.start()   # same options, same checkpoint
```

The docs' example takes a windowed count (a 10-minute watermark *≈ a rule telling Spark how late data may arrive, so it can finalize old windows and drop their state*, 5-minute windows grouped by `id`, written to a Delta table) from

the default 200 partitions down to 100.[23] In a Lakeflow pipeline you put the same key in `spark_conf`, per flow on `@dp.append_flow` or per table on `@dp.table`:[23]

Resizing state: before vs now

Before (open-source Spark)

```
# has no effect on an existing
# checkpoint: silently ignored
spark.conf.set(
  "spark.sql.shuffle.partitions",
  "100")
# real options: keep 200, or
# delete the checkpoint and
# lose all state
```

Now (Databricks, DBR 18+)

```
@dp.table(spark_conf={
  "spark.sql.streaming"
  ".stateStore.partitions":
    "100"})
def clicks_per_user():
    return (...)
# next run: state rehashed
# 200 -> 100, then resumes
```

Right pane follows the Lakeflow example in Databricks' state repartitioning docs; the function body is elided.

How long does it take? Databricks says "only a few seconds for most workloads", and that larger state means longer repartitions. You don't have to guess: after the next micro-batch, the query's progress event reports the time as `controlBatch.REPARTITION` under `durationMs`.[22,23]

🧪 Try it: find out what you're running with

Open-source Spark 4.0+ ships a "State metadata source" that reads a checkpoint as a table, including each stateful operator's `numPartitions`:[30]

```
(spark.read.format("state-metadata")
   .load("<checkpointLocation>")
   .select("operatorId", "operatorName",
           "numPartitions")
   .show())
```

It only works on checkpoints written by Spark 4.0+, and the related `statestore` reader (experimental) returns every key and value with its `partition_id`, which is a handy way to check for skew before you resize.[30]

WHAT IT MEANS FOR YOU

If you run stateful streams on Databricks, audit the partition counts. Spark's default is 200, and Databricks' general advice is "1-2 times the number of cores in the cluster."[25] A job whose cluster has grown or shrunk a lot since it started is a candidate. So is a tiny stream sitting on 200 partitions: each partition is a separate RocksDB instance in the checkpoint, so every micro-batch syncs state for every partition.[22,26]

Real-world evidence is one customer so far. Databricks' launch post quotes Coveo, an early adopter, which had been choosing between overprovisioning and rebuilding checkpoints, with storage API costs nearly as

high as compute costs. Coveo reports a 40% reduction in the related Amazon S3 API costs after resizing.[22] That's a customer claim published by the vendor, but the mechanism behind it (fewer partitions, fewer state files to write per batch) makes sense.

Use it for bursts too. The blog's use cases include scaling up for a historical backfill, then back down, and adapting to load that varies by time of day.[22] Each change is a stop and restart with a repartition pause, so it's a deliberate operation, not autoscaling.

If you're on open-source Spark, EMR or another platform, none of this applies yet. Your options are still the old ones: pick the count carefully at the start, or use **coalesce** *≈ a Spark operation that merges partitions into fewer tasks without a full reshuffle* to run fewer tasks for stateful operations without changing the state layout, which Spark's guide suggests.[24]

💥 Watch out for

- **RocksDB only.** Queries still on the older state store provider can't use it, and you can't switch the provider on an existing checkpoint: "you must restart it from scratch with a new checkpoint location to change the state store."[25,23]
- **"18 LTS" vs "18".** The repartitioning page says "Databricks Runtime 18 LTS and above", but the DBR 18 release notes say 18 "will transition to LTS when Databricks Runtime 19 is generally available."[23,21] In practice: DBR 18 or later.
- **More partitions don't fix a hot key.** One key always hashes to one partition. If one customer produces half your events, they stay on one task whatever the count.
- **The pause grows with state.** Test the resize on a copy of a large checkpoint before doing it to a production job with a tight latency SLA, and watch `controlBatch.REPARTITION`.
- **It's not upstream.** A pipeline you move off Databricks keeps the old Spark rule.[24]

WHAT HAPPENS NEXT

The obvious next question is whether this comes to open-source Spark. The stateless half already did, in 4.1.0.[27] The stateful half is Databricks-only for now, and nothing public says when or whether that changes. Meanwhile, the next thing to watch is whether Databricks makes resizing automatic, using the same repartition step without a manual stop and restart.

📖 Words from this story

Spark Structured Streaming: Spark's stream-processing engine, which runs queries as a series of small batch jobs.

Micro-batches: the small batch jobs that process data arrived since the previous one.

Stateless: processing each record on its own, without remembering earlier records.

State: the data a streaming query keeps between micro-batches, such as running counts per key.

Checkpoint: durable storage of a query's source progress and state, used to resume after a restart.

Shuffle: redistributing rows across the cluster so all rows with the same key reach the same task.

Shuffle partition count: how many buckets a shuffle uses (`spark.sql.shuffle.partitions` , default 200).

RocksDB: an embedded LSM-tree key-value store; Databricks' default state store since DBR 17.3.

Changelog checkpointing: writing only changed state records to the checkpoint each micro-batch.

Key groups: Flink's fixed slices of keyed state, reassigned between tasks when you rescale.

Lakeflow pipelines: Databricks' declarative pipeline framework for tables and flows.

transformWithState: Spark's newer API for custom per-key stateful logic.

Stream-static join: a join between a stream and an ordinary table; needs no state.

Watermark: how late data may arrive before Spark finalizes a window and drops its state.

Coalesce: merging partitions into fewer tasks without a full reshuffle.

On the radar

Also happened, in one line each. Some will get a full deep dive in a future issue.

- **PostgreSQL 19's schedule slipped:** the open-items wiki now lists RC1 on 15 October, an RC2 on 12 November and GA planned for 19 November; when we checked it for issue #8, it showed GA on 29 October.[31]
- **GuardDuty watches Postgres queries:** Amazon GuardDuty RDS Protection now analyzes queries on Aurora PostgreSQL and RDS for PostgreSQL to flag data exfiltration and destruction, using behavioral baselines and no agents.[32]
- **Lambda gets OAuth for Kafka:** Lambda event source mappings for self-managed Kafka (including Confluent Cloud, Aiven and Redpanda) can now authenticate through OAuth identity providers, beyond SASL/PLAIN, SASL/SCRAM and mTLS.[32]
- **Prime Day by AWS's numbers:** AWS says DynamoDB served more than 59 trillion requests over four days, peaking at 192 million requests per second.[33]
- **Valkey's next chapter:** Valkey maintainers Madelyn Olson and Viktor Söderqvist told InfoQ that synchronous replication and tiering hot data in memory with colder data on SSD are proposed directions.[34]
- **A DuckDB file with no data in it:** the DuckDB team showed how a tiny database holding only views over remote Parquet files works as a shareable, read-only catalog you attach by URL (7 Oct).[35]
- **Databricks (12 Oct):** Lakeflow pipeline parameters are GA, and Unity Catalog governed-tag propagation through lineage entered Beta.[36]

1. **MongoDB Launches MongoDB 9.0, the Best Version Ever Built, and Atlas Infinite for AI-Scale Demand**, MongoDB press release, 29 Sep 2026.
2. **MongoDB Queryable Encryption: Prefix, Suffix, and Substring Queries Are Now GA**, Alex Bauer and Cynthia Braund, MongoDB blog, 6 Oct 2026.
3. **Supported Operations for Queryable Encryption**, MongoDB Database Manual 9.0, viewed 14 Oct 2026.
4. **Encrypted Fields and Enabled Queries**, MongoDB Database Manual 9.0, viewed 14 Oct 2026.

5. **Queryable Encryption with Explicit Encryption**, MongoDB Database Manual 9.0, viewed 14 Oct 2026.
6. **Release Notes for MongoDB 8.2**, MongoDB Database Manual, viewed 14 Oct 2026.
7. **mongodb/node-mongodb-native v7.5.0**, GitHub release, 7 Jul 2026.
8. **Queryable Encryption Features**, MongoDB Database Manual 9.0, viewed 14 Oct 2026.
9. **Choosing an In-Use Encryption Approach**, MongoDB Database Manual 8.3, viewed 14 Oct 2026.
10. **The MongoDB Cryptography Research Group**, MongoDB, viewed 14 Oct 2026.
11. **Compatibility Changes in MongoDB 7.0**, MongoDB Database Manual.
12. **MongoDB Announces General Availability of End-To-End Data Encryption Technology**, MongoDB press release via Nasdaq, 15 Aug 2023.
13. **MongoDB Announces General Availability of MongoDB 8.0**, MongoDB press release via Nasdaq, 2 Oct 2024.
14. **Create an Encryption Schema**, MongoDB Database Manual 8.3, viewed 14 Oct 2026.
15. **Create an Encryption Schema**, MongoDB Database Manual 9.1 (upcoming), viewed 14 Oct 2026.
16. **Estimate Queryable Encryption Storage Impact**, MongoDB Database Manual 9.0, viewed 14 Oct 2026.
17. **Security Analysis of MongoDB Queryable Encryption**, Zichen Gui, Kenneth G. Paterson and Tianxin Tang, ETH Zurich, made public 5 Jun 2023.
18. **MongoDB 8.2's encrypted substring search costs 20 times more per insert than equality**, Alex Georgiev, DEV Community, 16 Sep (labelled AI-assisted).
19. **$encStrStartsWith (aggregation)**, MongoDB Database Manual 9.0, viewed 14 Oct 2026.
20. **Databricks platform release notes, October 2026**, Databricks, entry dated 13 Oct 2026.
21. **Databricks Runtime 18**, Databricks release notes, viewed 14 Oct 2026.
22. **Announcing On-Demand State Repartitioning for Apache Spark Structured Streaming on Databricks**, Thangam Vaiyapuri, Jay Palaniappan, B. Micheal Okutubo and Zifei Feng, Databricks blog, 14 Sep 2026.
23. **On-demand state repartitioning for stateful streaming queries**, Databricks documentation, viewed 14 Oct 2026.
24. **Structured Streaming Programming Guide: Additional Information**, Apache Spark 4.2.0 documentation.
25. **What is stateful streaming?**, Databricks documentation, viewed 14 Oct 2026.
26. **Configure RocksDB state store**, Databricks documentation, viewed 14 Oct 2026.
27. **SPARK-53942: Support changing stateless shuffle partitions upon restart of streaming query**, Apache Spark JIRA (issues list archive), resolved 25 Oct 2025.
28. **Stateful Stream Processing**, Apache Flink documentation.
29. **Parallel Execution**, Apache Flink documentation.
30. **State Data Source Integration Guide**, Apache Spark 4.2.0 documentation.
31. **PostgreSQL 19 Open Items**, PostgreSQL wiki, viewed 14 Oct 2026.
32. **AWS Weekly Roundup (October 13, 2026)**, AWS News Blog, 13 Oct 2026.
33. **AWS Weekly Roundup (October 13, 2026)**, AWS News Blog, 13 Oct 2026 (citing AWS's Prime Day 2026 metrics post).
34. **Olson and Söderqvist Discuss Valkey's Evolution and Future Past Caching Use Cases at OSS EU**, Olimpiu Pop, InfoQ, 9 Oct 2026.
35. **A DuckDB Database with No Data in It**, DuckDB, 7 Oct 2026.
36. **Databricks platform release notes, October 2026**, Databricks, entries dated 12 Oct 2026.